Chapter 17: Deploying ML Pipelines

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to design end-to-end ML pipeline architectures with separate training and inference code paths
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to implement ETL processes that extract, transform, and load data from operational systems into analytics-ready formats
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to serialize trained models using joblib and implement model versioning with training metadata
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to build inference pipelines that load saved models and generate predictions on new data in production environments
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to implement scheduled retraining workflows that maintain model currency over time

17.1Introduction

Figure 17.1:

What Deployment Really Means

In many courses, “deployment” is presented as something that happens only in the cloud—through external APIs, managed platforms, and specialized MLOps tools. Those approaches matter, but they are not where most real-world machine learning work begins.

In practice, the first version of a deployed model is often *embedded directly inside an application*. A team trains a model, saves it as a file, and loads it inside the same codebase that powers the product. The application reads fresh data from a database, runs the model to produce predictions, and logs results for monitoring and improvement. In practice, this often includes a lightweight analytical copy of operational data created through scheduled ETL, even when everything runs on the same machine.

This chapter teaches that foundational deployment pattern: an *end-to-end machine learning pipeline* that lives in your application environment. You will not use AWS, Azure, or external model-serving APIs here. Instead, you will build a complete pipeline that you can run on your own machine or in a controlled classroom environment.

This approach is intentionally simple. It is not the most scalable or the most sophisticated architecture—but it is *realistic*, *teachable*, and *extremely common* in early-stage products, internal analytics tools, and small-team deployments.

What This Chapter Builds

By the end of this chapter, you will be able to run a complete ML pipeline using a small set of Python scripts that do the following:

- Load data directly from a live operational database (the same database used by the application).

- Apply automated cleaning and feature engineering using reusable pipeline functions.
- Train a model and evaluate it using a standard train/test workflow.
- Save the trained model to disk as a versioned file (for example, a *.sav* artifact).
- Schedule retraining to run automatically (for example, nightly or weekly).
- Load the saved model inside an application-like script and run predictions on new records.

This is what “deployment” often looks like before a team invests in data warehouses, orchestration platforms, and enterprise MLOps systems. Understanding this foundation will make advanced tools easier to learn later because you will know what those tools are automating.

Key Mental Model: Deployment Is About Reliability

A deployed model is not defined by where it runs. It is defined by whether it can run *reliably* and *repeatably* in a real process that other people or systems depend on.

This chapter emphasizes practical reliability principles that apply in every environment—from a single Python script to enterprise cloud platforms:

- **Repeatability:** the same pipeline produces the same outputs when run on the same inputs.
- **Traceability:** you can identify which code, data, and model version produced a prediction.

- **Separation of concerns:** training code, inference code, and data access code are organized clearly.
- **Safe failure:** the system fails gracefully (with clear logging) instead of silently producing unreliable outputs.

Why We Start Here (Before Cloud Platforms)

Cloud platforms like Azure ML Studio and managed MLOps pipelines are valuable, but they can hide the basic moving parts. In this chapter, you will build the moving parts yourself so you understand what is happening at each step.

Later, when you use managed tools, you will recognize the same pipeline components—data loading, cleaning, training, evaluation, model registration, scheduling, and monitoring—just wrapped in a larger system.

Hands-On Preview: The Simplest “Deployed Model”

To preview what deployment means in this chapter, here is the smallest possible example. We train a model, save it as a file, and load it later to make predictions. You will build a more complete version of this workflow throughout the chapter.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
import joblib

# 1. Train a small model (toy example)
X, y = make_classification(n_samples=500, n_features=8, random_state=42)
model = LogisticRegression(max_iter=500)
model.fit(X, y)

# 2. Save the model artifact to disk
joblib.dump(model, &amp;quot;model.sav&amp;quot;)
```

```
# 3. Load the model artifact later (simulating an application restart)
loaded = joblib.load(&amp;quot;model.sav&amp;quot;)

# 4. Run inference on new data
x_new = np.random.rand(1, 8)
pred = loaded.predict(x_new)
proba = loaded.predict_proba(x_new)

print(&amp;quot;Prediction:&amp;quot;, int(pred[0]))
print(&amp;quot;Probabilities:&amp;quot;, proba[0])

# Output:
```

This example demonstrates a core deployment truth: a trained model is just an artifact that can be persisted and reused. The rest of deployment work is about building a stable pipeline around that artifact, ensuring that the data and transformations used during training match what will happen in production.

In the next section, you will see the full embedded deployment architecture and how all parts of the pipeline connect: application, operational database, training script, model artifact, retraining schedule, and inference process.

17.2Deployment Architecture

A Simple but Realistic ML Deployment Architecture

In this chapter, “deployment” means connecting your model to a real application workflow: data flows into a live database, a scheduled training script rebuilds the model periodically, the model is saved to disk, and the application loads that saved model to make predictions on new records.

This design is intentionally simple: the training process reads directly from the same operational database used by the application. In production, many organizations insert an analytics layer (a warehouse or lakehouse) between

the operational database and modeling, but this streamlined version is realistic for small teams and many early-stage products. *This approach still allows for lightweight analytical copies of the data created via scheduled ETL without introducing full-scale infrastructure.*

Core Components

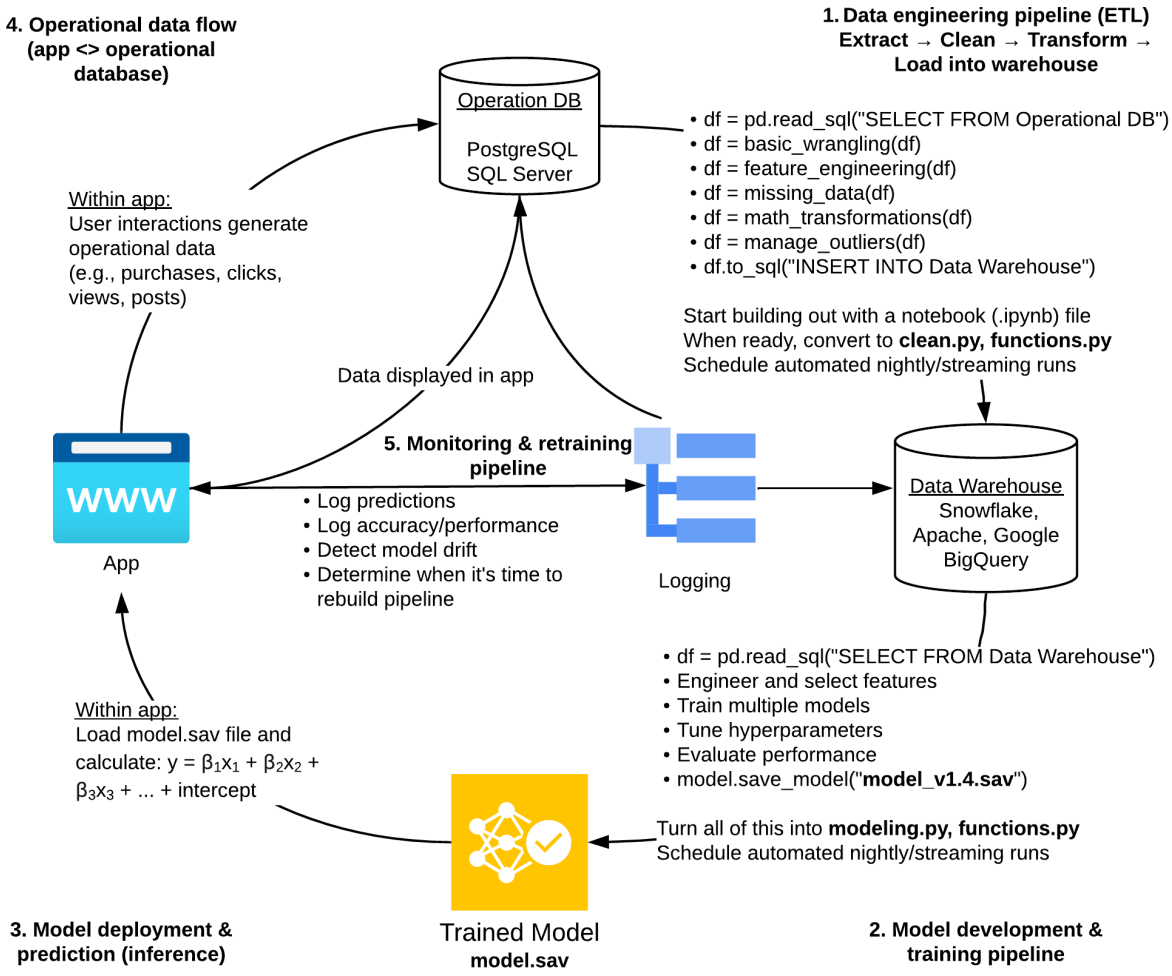
- **Application**: collects inputs, writes new records, and requests predictions during user workflows.
- **Operational database**: the live system of record (for example, PostgreSQL, MySQL, SQL Server) storing application data.
- **Periodic training script**: a Python program that queries the database, cleans data, trains a model, evaluates it, and saves artifacts.
- **Saved model file**: a serialized artifact (for example, a *.sav* file) that the application can load for inference.
- **Inference code path**: application logic that loads the latest model and produces predictions for new inputs.

End-to-End Data Flow

1. The **application** writes new transactions and events into the **operational database**.
2. On a schedule (for example, nightly), a **training script** reads the latest data from the database.
3. The script runs **the same cleaning and feature engineering logic** you used during modeling, rather than re-implementing transformations separately for training and inference.

4. The script trains and evaluates a model, then saves the **model artifact** (and often a preprocessing artifact) to a known location.
5. The **application** loads the newest saved model and uses it to generate predictions during normal workflows.

Architecture Diagram



Where Warehouses and Lakehouses Fit Later

At larger scale, teams often separate operational and analytics workloads by copying data into a warehouse or lakehouse (for example, Snowflake or a

Spark-based platform). That separation improves performance, governance, reproducibility, and historical tracking, but the core pipeline steps you practice here still apply.

17.3End-to-End Pipeline

Designing the End-to-End Pipeline

An end-to-end machine learning deployment pipeline is simply a structured way to turn the CRISP-DM process into executable code. Every stage you learned earlier—understanding data, preparing it, modeling, and evaluation—still exists. The difference is that these steps now run automatically and repeatedly.

Seen this way, deployment is not a new discipline. It is *CRISP-DM operationalized*.

Pipeline Stages

A well-designed pipeline breaks the workflow into clear, testable stages. Each stage should be implemented as a function or module that can be reused, logged, and debugged independently. In practice, early stages are often executed as a lightweight ETL process that prepares analytical-ready data for modeling.

- **Data ingestion**: Connect to the operational database, query relevant records, and load them into a working DataFrame. This step defines the snapshot of data used for training.
- **Automated cleaning**: Apply the same reusable cleaning functions developed earlier in the course (wrangling, dates, bins, missing data,

outliers). No dataset-specific logic should appear beyond this stage.

- **Feature engineering**: Transform cleaned data into model-ready features, including encoding, scaling, and derived variables. These transformations must be identical during training and inference, ideally enforced through shared pipeline code.
- **Model training**: Fit the selected algorithm using the engineered features. This step mirrors traditional supervised learning and should include clear configuration of hyperparameters.
- **Evaluation**: Measure performance using a validation or holdout set. Metrics should be logged and compared over time to detect degradation or improvement.
- **Model serialization**: Save the trained model (and any required preprocessing objects) to disk in a standardized format such as *.sav*. This artifact is what the application will load for inference.

Why This Structure Matters

Separating the pipeline into stages enforces discipline. Each step has a single responsibility, which makes the system easier to test, modify, and explain.

Most deployment failures are not modeling failures—they are pipeline failures. Clear boundaries and reusable components reduce the risk of data leakage, inconsistent preprocessing, and silent behavior changes.

CRISP-DM Turned Into Code

If this structure feels familiar, it should. The pipeline maps directly to CRISP-DM:

- Business understanding → defining the prediction task and evaluation criteria
- Data understanding → ingestion and basic validation
- Data preparation → automated cleaning and feature engineering
- Modeling → training and tuning
- Evaluation → metric computation and comparison
- Deployment → serialization and integration with the application

The core insight is simple but powerful: deployment does not replace analytics fundamentals—it forces you to apply them consistently, every time the pipeline runs.

17.4ETL

In real-world systems, machine learning models are rarely trained directly on live operational databases. Instead, data is extracted, transformed, and loaded (ETL) into a separate analytical store that is optimized for modeling.

In this chapter, we simulate that production pattern using two SQLite databases:

This download can be found online.

This download can be found online.

- *shop.db* — the live operational database used by the application
- *warehouse.db* — a denormalized analytical database used for modeling

This separation reinforces an important deployment principle: models should train on stable, well-structured data, not directly on transactional tables. The ETL process can be run repeatedly on a schedule, producing the same analytical output given the same source data.

What This ETL Script Does

The ETL process in this section performs four core steps:

- Extract relevant tables from the operational database
- Join and denormalize the data into a single modeling table
- Perform light, repeatable cleaning and feature construction
- Load the result into a separate SQLite database

All dataset-specific logic remains here. Downstream modeling code will assume that the data is already clean, consistent, and modeling-ready.

Extract and Join Operational Data

We begin by connecting to the operational database and loading the tables needed for predicting late delivery.

```
import sqlite3
import pandas as pd

# Connect to operational database
conn = sqlite3.connect(&amp;quot;shop.db&amp;quot;)

# Load core tables
orders = pd.read_sql(&amp;quot;SELECT * FROM orders&amp;quot;, conn)
customers = pd.read_sql(&amp;quot;SELECT * FROM customers&amp;quot;, conn)
order_items = pd.read_sql(&amp;quot;SELECT * FROM order_items&amp;quot;, conn)
products = pd.read_sql(&amp;quot;SELECT * FROM products&amp;quot;, conn)

conn.close()

print(orders.shape, customers.shape, order_items.shape, products.shape)
```

At this stage, the data is highly normalized and not suitable for modeling. Our goal is to collapse these tables into one row per order.

Denormalize to One Row per Order

We first aggregate order-level features from the order items table.

```
# Aggregate order items
order_item_features = (
  order_items
    .merge(products, on=&quot;product_id&quot;, how=&quot;left&quot;)
    .groupby(&quot;order_id&quot;)
    .agg(
      num_items=(&quot;quantity&quot;, &quot;sum&quot;),
      avg_price=(&quot;price&quot;, &quot;mean&quot;),
      total_value=(&quot;price&quot;, &quot;sum&quot;),
      avg_weight=(&quot;weight&quot;, &quot;mean&quot;)
    )
    .reset_index()
)

order_item_features.head()
```

Next, we join customers, orders, and aggregated item features into a single table.

```
# Join everything into one modeling table
df = (
  orders
    .merge(customers, on=&quot;customer_id&quot;, how=&quot;left&quot;)
    .merge(order_item_features, on=&quot;order_id&quot;, how=&quot;left&quot;)
)

df.head()
```

Feature Engineering for Late Delivery

We now create a small number of transparent features that plausibly influence delivery delays. These features are deterministic and reproducible, making them safe to use in both training and inference.

```
      # Convert dates
      df[&amp;quot;order_date&amp;quot;] = pd.to_datetime(df[&amp;quot;order_date&amp;quot;])
      df[&amp;quot;ship_date&amp;quot;] = pd.to_datetime(df[&amp;quot;ship_date&amp;quot;])

      # Delivery time (used only to define the label)
      df[&amp;quot;delivery_days&amp;quot;] = (df[&amp;quot;ship_date&amp;quot;] -
df[&amp;quot;order_date&amp;quot;]).dt.days

      # Customer age
      df[&amp;quot;birthdate&amp;quot;] = pd.to_datetime(df[&amp;quot;birthdate&amp;quot;])
      df[&amp;quot;customer_age&amp;quot;] = (df[&amp;quot;order_date&amp;quot;] -
df[&amp;quot;birthdate&amp;quot;]).dt.days // 365

      # Historical order volume per customer
      df[&amp;quot;customer_order_count&amp;quot;] = (
        df.groupby(&amp;quot;customer_id&amp;quot;)[&amp;quot;order_id&amp;quot;]
          .transform(&amp;quot;count&amp;quot;)
      )

      df[[
        &amp;quot;num_items&amp;quot;,
        &amp;quot;total_value&amp;quot;,
        &amp;quot;avg_weight&amp;quot;,
        &amp;quot;customer_age&amp;quot;,
        &amp;quot;customer_order_count&amp;quot;
      ]].describe()
```

Notice that no model logic appears here. This script constructs features and labels only; learning happens later.

Define the Modeling Target

For this chapter, we predict whether an order is delivered late using a simple business rule.

```
      # Define target: late delivery (1 = late, 0 = on time)
      df[&amp;quot;late_delivery&amp;quot;] = (df[&amp;quot;delivery_days&amp;quot;] &amp;gt; 5).astype(int)

      df[&amp;quot;late_delivery&amp;quot;].value_counts(normalize=True)
```

The variable *delivery_days* is used only to construct the label and is intentionally excluded from modeling features to avoid data leakage.

Load into the Analytical Database

Finally, we write the denormalized dataset into a separate SQLite database used exclusively for modeling.

```
# Connect to analytical database
warehouse_conn = sqlite3.connect(&amp;quot;warehouse.db&amp;quot;)

# Write modeling table
df.to_sql(
  &amp;quot;fact_orders_ml&amp;quot;,
  warehouse_conn,
  if_exists=&amp;quot;replace&amp;quot;,
  index=False
)

warehouse_conn.close()

print(&amp;quot;warehouse.db created with table: fact_orders_ml&amp;quot;)
```

Key Takeaways

This ETL step turns raw application data into a stable analytical asset.

Once created, *warehouse.db* becomes the sole data source for training and evaluation, forming a clear contract between data engineering and modeling.

This mirrors real deployment pipelines: operational systems feed analytics systems, which feed models.

In the next section, you will build a training pipeline that assumes this data already exists and focuses entirely on modeling.

17.5Training

With the ETL pipeline complete, we can now focus entirely on model training. At this stage, the problem looks like any other supervised learning task you have seen earlier in this course.

This separation is intentional. The training code assumes that data cleaning and feature construction have already been handled upstream in the warehouse build step and does not reach back into the operational database.

Load the Modeling Data

We begin by loading the denormalized modeling table from the analytical database (the warehouse SQLite file). Each row represents one order, with engineered features and a labeled target.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(&amp;quot;warehouse.db&amp;quot;)

# Load the modeling table created by the ETL step
df = pd.read_sql(&amp;quot;SELECT * FROM fact_orders_ml&amp;quot;, conn)
conn.close()

print(df.shape)
df.head()
```

Select Features and Target

We explicitly define the columns used for modeling. This makes the pipeline transparent and ensures the same features are used during both training and inference.

```
from sklearn.model_selection import train_test_split

label_col = &amp;quot;late_delivery&amp;quot;

feature_cols = [
  &amp;quot;num_items&amp;quot;,
  &amp;quot;total_value&amp;quot;,
  &amp;quot;avg_weight&amp;quot;,
  &amp;quot;customer_age&amp;quot;,
  &amp;quot;customer_order_count&amp;quot;
]

X = df[feature_cols]
y = df[label_col].astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split(
  X, y,
  test_size=0.25,
  random_state=42,
  stratify=y
)

X_train.shape, X_test.shape
```

Notice what is missing here: we do not perform ad-hoc cleaning or feature creation inside the training step. That work belongs in ETL so that training remains consistent, repeatable, and auditable.

Build a Training Pipeline

We now introduce scikit-learn's *Pipeline* object. A pipeline packages preprocessing and the model into a single unit that can be saved to disk and reused during inference.

This is a deployment best practice: the exact same transformations must be applied at training time and prediction time.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline(steps=[
  (&amp;quot;imputer&amp;quot;, SimpleImputer(strategy=&amp;quot;median&amp;quot;)),
  (&amp;quot;scaler&amp;quot;, StandardScaler()),
  (&amp;quot;model&amp;quot;, LogisticRegression(max_iter=1000))
])

pipeline
```

Train the Model

Training becomes a single method call. The pipeline learns both preprocessing parameters (imputation values and scaling parameters) and the model weights.

```
pipeline.fit(X_train, y_train)
```

Evaluate Performance

Before saving the model, evaluate on unseen data. Even in deployment contexts, this remains standard supervised learning: test sets still matter.

```
from sklearn.metrics import classification_report, accuracy_score, f1_score, roc_auc_score

y_pred = pipeline.predict(X_test)
y_prob = pipeline.predict_proba(X_test)[:, 1]

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_prob)

report = classification_report(y_test, y_pred, output_dict=True)

accuracy, f1, roc_auc
```

In production systems, these metrics should always be recorded alongside the model so performance can be compared across retraining runs.

Save the Model Artifact

A trained model is just a file. We serialize the entire pipeline using *joblib*, which preserves preprocessing and modeling logic together.

```
import joblib

joblib.dump(pipeline, &amp;quot;late_delivery_model.sav&amp;quot;)
```

This single file can now be loaded by any Python process for inference. That is why we say: *Your model is a file.*

Save Metrics and Metadata

Production models should always be accompanied by metadata (training context) and metrics (evaluation results). These files make the model auditable, traceable, and reproducible.

```
import json
from datetime import datetime

model_version = &amp;quot;1.0.0&amp;quot;

metadata = {
  &amp;quot;model_name&amp;quot;: &amp;quot;late_delivery_pipeline&amp;quot;,
  &amp;quot;model_version&amp;quot;: model_version,
  &amp;quot;trained_at_utc&amp;quot;: datetime.utcnow().isoformat(),
  &amp;quot;warehouse_table&amp;quot;: &amp;quot;fact_orders_ml&amp;quot;,
  &amp;quot;num_training_rows&amp;quot;: int(X_train.shape[0]),
  &amp;quot;num_test_rows&amp;quot;: int(X_test.shape[0]),
  &amp;quot;features&amp;quot;: feature_cols
}

metrics = {
  &amp;quot;accuracy&amp;quot;: float(accuracy),
  &amp;quot;f1&amp;quot;: float(f1),
  &amp;quot;roc_auc&amp;quot;: float(roc_auc),
  &amp;quot;classification_report&amp;quot;: report
}

with open(&amp;quot;model_metadata.json&amp;quot;, &amp;quot;w&amp;quot;, encoding=&amp;quot;utf-
8&amp;quot;) as f:
  json.dump(metadata, f, indent=2)

with open(&amp;quot;metrics.json&amp;quot;, &amp;quot;w&amp;quot;, encoding=&amp;quot;utf-8&amp;quot;) as f:
  json.dump(metrics, f, indent=2)
```

What Gets Deployed

In this architecture, deployment means the application can reliably access the following artifacts:

- *late_delivery_model.sav* — the trained pipeline (preprocessing + model)
- *model_metadata.json* — version, timestamp, row counts, and feature list
- *metrics.json* — evaluation metrics and classification report

The scheduler may retrain this model on a schedule, but the application does not retrain. The application simply loads the latest model file or consumes

predictions written back to the database.

Key Idea

Once the ETL step produces a consistent modeling table, training becomes fully repeatable and automatable.

Your model is a file.

In the next section, you will load this saved pipeline, generate late-delivery probabilities for live orders, and write predictions back into the operational database to support a warehouse priority workflow.

17.6Inference

Once a model has been trained and saved, deployment shifts from learning to prediction. This phase is called *inference*.

Inference does not involve training, gradients, or optimization. The model is loaded from disk and used to generate predictions on new data.

Load the Trained Model

We begin by loading the serialized model artifact produced during training.

```
import joblib

model = joblib.load(&amp;quot;late_delivery_model.sav&amp;quot;)
model
```

This object contains both preprocessing steps and the trained classifier.

Load New Orders from the Operational Database

Predictions are generated on live operational data, not the analytical warehouse.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(&amp;quot;shop.db&amp;quot;)

query = &amp;quot;&amp;quot;&amp;quot;
SELECT
  o.order_id,
  o.num_items,
  o.total_value,
  o.avg_weight,
  o.order_date,
  c.birthdate,
  c.customer_id
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.fulfilled = 0
&amp;quot;&amp;quot;&amp;quot;

df_live = pd.read_sql(query, conn)
df_live.head()
```

These orders have not yet been fulfilled and may benefit from prioritization.

Feature Engineering at Inference Time

Inference must apply the *same feature logic* used during training. In production systems, this logic should be shared code imported by both training and inference scripts.

```
from datetime import datetime

# Customer age (consistent with ETL logic)
df_live[&amp;quot;birthdate&amp;quot;] = pd.to_datetime(df_live[&amp;quot;birthdate&amp;quot;])
df_live[&amp;quot;order_date&amp;quot;] = pd.to_datetime(df_live[&amp;quot;order_date&amp;quot;])
df_live[&amp;quot;customer_age&amp;quot;] = (
  (df_live[&amp;quot;order_date&amp;quot;] - df_live[&amp;quot;birthdate&amp;quot;]).dt.days // 365
)

# Historical order count per customer
order_counts = (
  df_live.groupby(&amp;quot;customer_id&amp;quot;)[&amp;quot;order_id&amp;quot;]
    .transform(&amp;quot;count&amp;quot;)
)

df_live[&amp;quot;customer_order_count&amp;quot;] = order_counts
```

```
feature_cols = [
  &amp;quot;num_items&amp;quot;,
  &amp;quot;total_value&amp;quot;,
  &amp;quot;avg_weight&amp;quot;,
  &amp;quot;customer_age&amp;quot;,
  &amp;quot;customer_order_count&amp;quot;
]

X_live = df_live[feature_cols]
```

This step reinforces why pipelines and shared feature logic are critical in deployment.

Generate Predictions

We now use the trained model to predict late delivery risk.

```
df_live[&amp;quot;late_delivery_prob&amp;quot;] = model.predict_proba(X_live)[:, 1]
df_live[&amp;quot;late_delivery_pred&amp;quot;] = model.predict(X_live)

df_live[[&amp;quot;order_id&amp;quot;, &amp;quot;late_delivery_prob&amp;quot;,
&amp;quot;late_delivery_pred&amp;quot;]].head()
```

Probabilities are often more useful than binary predictions for operational decision-making.

Write Predictions Back to the Operational Database

Predictions become most valuable when written back into systems that drive action.

```
cursor = conn.cursor()

cursor.execute(&amp;quot;&amp;quot;&amp;quot;
CREATE TABLE IF NOT EXISTS order_predictions (
  order_id INTEGER PRIMARY KEY,
  late_delivery_probability REAL,
  predicted_late_delivery INTEGER,
  prediction_timestamp TEXT
)
&amp;quot;&amp;quot;&amp;quot;)

rows = [
```

```
  (
    int(row.order_id),
    float(row.late_delivery_prob),
    int(row.late_delivery_pred),
    datetime.utcnow().isoformat()
  )
  for row in df_live.itertuples()
]

cursor.executemany(&amp;quot;&amp;quot;&amp;quot;
INSERT OR REPLACE INTO order_predictions
(order_id, late_delivery_probability, predicted_late_delivery, prediction_timestamp)
VALUES (?, ?, ?, ?)
&amp;quot;&amp;quot;&amp;quot;, rows)

conn.commit()
conn.close()
```

The operational database now contains model output alongside transactional data.

Using Predictions in the Application

At this point, the web application does not need to know anything about machine learning.

It simply queries a table that already contains predictions.

```
SELECT *
FROM order_predictions
ORDER BY late_delivery_probability DESC;
```

This enables a warehouse dashboard to prioritize orders most likely to be delayed.

Key Deployment Pattern

- Models are trained offline
- Predictions are written into operational systems
- Applications consume predictions like any other data

This pattern avoids embedding ML logic directly into application code.

Key Idea

Inference is not about intelligence. It is about integration.

A deployed model is useful only when its predictions influence real decisions.

In the final section, we will reflect on how this pipeline mirrors real-world deployment—and when simpler solutions are preferable.

17.7Scheduled Jobs

So far, you have developed the pipeline in notebooks so you can experiment, inspect intermediate outputs, and debug quickly. In real deployments, however, these same steps must run automatically on a schedule.

In this chapter, we will convert the core logic into a small set of Python scripts (.py files). These scripts will be executed in a repeating cycle:

- **ETL** creates a denormalized modeling table in a separate SQLite warehouse file.
- **Training** trains a model from the warehouse table, saves the model file, and saves metadata and metrics.
- **Inference** loads the saved model and writes predictions back into the operational database for the application to use.

This structure helps you see the difference between analytics work (in notebooks) and production work (scheduled jobs). Each job is reusable code that can be run manually for debugging or run automatically for deployment.

Project Folder Layout

Create a folder for your project with the following structure:

```
project/
  data/
    shop.db
    warehouse.db
  artifacts/
    late_delivery_model.sav
    model_metadata.json
    metrics.json
  jobs/
    config.py
    utils_db.py
    etl_build_warehouse.py
    train_model.py
    run_inference.py
```

The **data** folder holds your operational database (shop.db) and your simplified warehouse database (warehouse.db). The **artifacts** folder holds outputs produced by training.

Shared Configuration

All jobs should agree on paths and filenames. Put shared paths in one place, and make sure the artifacts folder exists before saving outputs.

```
# jobs/config.py
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]

DATA_DIR = PROJECT_ROOT / &amp;quot;data&amp;quot;
ARTIFACTS_DIR = PROJECT_ROOT / &amp;quot;artifacts&amp;quot;

OP_DB_PATH = DATA_DIR / &amp;quot;shop.db&amp;quot;
WH_DB_PATH = DATA_DIR / &amp;quot;warehouse.db&amp;quot;

MODEL_PATH = ARTIFACTS_DIR / &amp;quot;late_delivery_model.sav&amp;quot;
MODEL_METADATA_PATH = ARTIFACTS_DIR / &amp;quot;model_metadata.json&amp;quot;
METRICS_PATH = ARTIFACTS_DIR / &amp;quot;metrics.json&amp;quot;
```

Database Utilities

These helpers keep database code consistent across ETL, training, and inference.

```
# jobs/utils_db.py
import sqlite3
from contextlib import contextmanager

@contextmanager
def sqlite_conn(db_path):
  conn = sqlite3.connect(str(db_path))
  try:
    yield conn
  finally:
    conn.close()

def ensure_predictions_table(conn):
  cur = conn.cursor()
  cur.execute(&amp;quot;&amp;quot;&amp;quot;
  CREATE TABLE IF NOT EXISTS order_predictions (
    order_id INTEGER PRIMARY KEY,
    late_delivery_probability REAL,
    predicted_late_delivery INTEGER,
    prediction_timestamp TEXT
  )
  &amp;quot;&amp;quot;&amp;quot;)
  conn.commit()
```

Job 1: ETL to Build the Warehouse

This job reads operational tables and writes a denormalized modeling table into a separate SQLite database (warehouse.db). If your table or column names differ, adjust the SQL query so it matches your shop.db schema.

```
# jobs/etl_build_warehouse.py
import pandas as pd
from datetime import datetime
from config import OP_DB_PATH, WH_DB_PATH
from utils_db import sqlite_conn

def build_modeling_table():
  with sqlite_conn(OP_DB_PATH) as conn:
    query = &amp;quot;&amp;quot;&amp;quot;
    SELECT
      o.order_id,
      o.customer_id,
```

```
            o.num_items,
            o.total_value,
            o.avg_weight,
            o.order_timestamp,
            o.late_delivery AS label_late_delivery,
            c.gender,
            c.birthdate
          FROM orders o
          JOIN customers c ON o.customer_id = c.customer_id
          &amp;quot;&amp;quot;&amp;quot;
          df = pd.read_sql(query, conn)

        df[&amp;quot;order_timestamp&amp;quot;] = pd.to_datetime(df[&amp;quot;order_timestamp&amp;quot;],
errors=&amp;quot;coerce&amp;quot;)
        df[&amp;quot;birthdate&amp;quot;] = pd.to_datetime(df[&amp;quot;birthdate&amp;quot;],
errors=&amp;quot;coerce&amp;quot;)

        # Feature engineering kept simple and repeatable
        now_year = datetime.now().year
        df[&amp;quot;customer_age&amp;quot;] = now_year - df[&amp;quot;birthdate&amp;quot;].dt.year

        df[&amp;quot;order_dow&amp;quot;] = df[&amp;quot;order_timestamp&amp;quot;].dt.dayofweek
        df[&amp;quot;order_month&amp;quot;] = df[&amp;quot;order_timestamp&amp;quot;].dt.month

        modeling_cols = [
          &amp;quot;order_id&amp;quot;,
          &amp;quot;customer_id&amp;quot;,
          &amp;quot;num_items&amp;quot;,
          &amp;quot;total_value&amp;quot;,
          &amp;quot;avg_weight&amp;quot;,
          &amp;quot;customer_age&amp;quot;,
          &amp;quot;order_dow&amp;quot;,
          &amp;quot;order_month&amp;quot;,
          &amp;quot;label_late_delivery&amp;quot;
        ]

        df_model = df[modeling_cols].dropna(subset=[&amp;quot;label_late_delivery&amp;quot;])

        with sqlite_conn(WH_DB_PATH) as wh_conn:
          df_model.to_sql(&amp;quot;modeling_orders&amp;quot;, wh_conn,
if_exists=&amp;quot;replace&amp;quot;, index=False)

        return len(df_model)

      if __name__ == &amp;quot;__main__&amp;quot;:
        row_count = build_modeling_table()
        print(f&amp;quot;Warehouse updated. modeling_orders rows: {row_count}&amp;quot;)
```

Job 2: Train the Model and Save Artifacts

This job trains the model from the warehouse table and writes three outputs to disk:

- **late_delivery_model.sav** (the trained model file)

- **model_metadata.json** (version, timestamp, row counts, feature list)
- **metrics.json** (evaluation metrics)

```
      # jobs/train_model.py
      import json
      from datetime import datetime
      import pandas as pd
      import joblib

      from sklearn.model_selection import train_test_split
      from sklearn.pipeline import Pipeline
      from sklearn.compose import ColumnTransformer
      from sklearn.preprocessing import OneHotEncoder
      from sklearn.impute import SimpleImputer
      from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
      from sklearn.linear_model import LogisticRegression

      from config import WH_DB_PATH, ARTIFACTS_DIR, MODEL_PATH, MODEL_METADATA_PATH,
METRICS_PATH
      from utils_db import sqlite_conn

      MODEL_VERSION = &quot;1.0.0&quot;

      def train_and_save():
        with sqlite_conn(WH_DB_PATH) as conn:
          df = pd.read_sql(&quot;SELECT * FROM modeling_orders&quot;, conn)

        label_col = &quot;label_late_delivery&quot;

        feature_cols = [c for c in df.columns if c != label_col]
        X = df[feature_cols]
        y = df[label_col].astype(int)

        X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, random_state=42, stratify=y
        )

        numeric_features = [&quot;num_items&quot;, &quot;total_value&quot;,
&quot;avg_weight&quot;, &quot;customer_age&quot;, &quot;order_dow&quot;,
&quot;order_month&quot;]
        categorical_features = []

        numeric_pipe = Pipeline(steps=[
          (&quot;imputer&quot;, SimpleImputer(strategy=&quot;median&quot;))
        ])

        categorical_pipe = Pipeline(steps=[
          (&quot;imputer&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
          (&quot;onehot&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;))
        ])

        preprocessor = ColumnTransformer(
          transformers=[
            (&quot;num&quot;, numeric_pipe, numeric_features),
            (&quot;cat&quot;, categorical_pipe, categorical_features)
          ],
          remainder=&quot;drop&quot;
        )
```

```
    clf = LogisticRegression(max_iter=500)

    model = Pipeline(steps=[
      (&amp;quot;prep&amp;quot;, preprocessor),
      (&amp;quot;clf&amp;quot;, clf)
    ])

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    metrics = {
      &amp;quot;accuracy&amp;quot;: float(accuracy_score(y_test, y_pred)),
      &amp;quot;f1&amp;quot;: float(f1_score(y_test, y_pred)),
      &amp;quot;roc_auc&amp;quot;: float(roc_auc_score(y_test, y_prob)),
      &amp;quot;row_count_train&amp;quot;: int(len(X_train)),
      &amp;quot;row_count_test&amp;quot;: int(len(X_test))
    }

    ARTIFACTS_DIR.mkdir(parents=True, exist_ok=True)

    joblib.dump(model, str(MODEL_PATH))

    metadata = {
      &amp;quot;model_version&amp;quot;: MODEL_VERSION,
      &amp;quot;trained_at_utc&amp;quot;: datetime.utcnow().isoformat(),
      &amp;quot;feature_list&amp;quot;: feature_cols,
      &amp;quot;label&amp;quot;: label_col,
      &amp;quot;warehouse_table&amp;quot;: &amp;quot;modeling_orders&amp;quot;,
      &amp;quot;warehouse_rows&amp;quot;: int(len(df))
    }

    with open(MODEL_METADATA_PATH, &amp;quot;w&amp;quot;, encoding=&amp;quot;utf-8&amp;quot;) as f:
      json.dump(metadata, f, indent=2)

    with open(METRICS_PATH, &amp;quot;w&amp;quot;, encoding=&amp;quot;utf-8&amp;quot;) as f:
      json.dump(metrics, f, indent=2)

    print(&amp;quot;Training complete.&amp;quot;)
    print(f&amp;quot;Saved model: {MODEL_PATH}&amp;quot;)
    print(f&amp;quot;Saved metadata: {MODEL_METADATA_PATH}&amp;quot;)
    print(f&amp;quot;Saved metrics: {METRICS_PATH}&amp;quot;)

if __name__ == &amp;quot;__main__&amp;quot;:
    train_and_save()
```

Job 3: Run Inference and Write Predictions to shop.db

This job loads the latest saved model and generates predictions for unfulfilled orders. It then writes predictions to a dedicated table keyed by order_id.

```
      # jobs/run_inference.py
      import pandas as pd
      import joblib
      from datetime import datetime

      from config import OP_DB_PATH, MODEL_PATH
      from utils_db import sqlite_conn, ensure_predictions_table

      def run_inference():
        model = joblib.load(str(MODEL_PATH))

        with sqlite_conn(OP_DB_PATH) as conn:
          query = &quot;&quot;&quot;
          SELECT
            o.order_id,
            o.num_items,
            o.total_value,
            o.avg_weight,
            o.order_timestamp,
            c.birthdate
          FROM orders o
          JOIN customers c ON o.customer_id = c.customer_id
          WHERE o.fulfilled = 0
          &quot;&quot;&quot;
          df_live = pd.read_sql(query, conn)

        df_live[&quot;order_timestamp&quot;] =
pd.to_datetime(df_live[&quot;order_timestamp&quot;], errors=&quot;coerce&quot;)
        df_live[&quot;birthdate&quot;] = pd.to_datetime(df_live[&quot;birthdate&quot;],
errors=&quot;coerce&quot;)

        now_year = datetime.now().year
        df_live[&quot;customer_age&quot;] = now_year -
df_live[&quot;birthdate&quot;].dt.year

        df_live[&quot;order_dow&quot;] = df_live[&quot;order_timestamp&quot;].dt.dayofweek
        df_live[&quot;order_month&quot;] = df_live[&quot;order_timestamp&quot;].dt.month

        X_live = df_live[[&quot;num_items&quot;, &quot;total_value&quot;,
&quot;avg_weight&quot;, &quot;customer_age&quot;, &quot;order_dow&quot;,
&quot;order_month&quot;]]

        probs = model.predict_proba(X_live)[:, 1]
        preds = model.predict(X_live)

        ts = datetime.utcnow().isoformat()
        out_rows = [(int(oid), float(p), int(yhat), ts) for oid, p, yhat in
zip(df_live[&quot;order_id&quot;], probs, preds)]

        with sqlite_conn(OP_DB_PATH) as conn:
          ensure_predictions_table(conn)
          cur = conn.cursor()
          cur.executemany(&quot;&quot;&quot;
          INSERT OR REPLACE INTO order_predictions
          (order_id, late_delivery_probability, predicted_late_delivery,
prediction_timestamp)
          VALUES (?, ?, ?, ?)
          &quot;&quot;&quot;, out_rows)
          conn.commit()

        print(f&quot;Inference complete. Predictions written: {len(out_rows)}&quot;)
```

```
if __name__ == &quot;__main__&quot;:
  run_inference()
```

How These Scripts Run Automatically

In production, these scripts are executed by a scheduler. The scheduler is not “AI.” It is simply a timed process that runs commands repeatedly.

A realistic schedule for this project could be:

- ETL runs every night (build modeling table)
- Training runs every night after ETL
- Inference runs every few minutes (keep predictions fresh)

Cron Scheduling Example (Mac or Linux)

Cron is a common scheduler on Unix-like systems. You edit your scheduled commands using:

```
crontab -e
```

Cron runs in a minimal environment. Use absolute paths and, if you are using a virtual environment, activate it explicitly in the command.

```
# Nightly ETL at 1:00am
0 1 * * * cd /path/to/project &amp;&amp; /path/to/venv/bin/python
jobs/etl_build_warehouse.py &gt;&gt; logs/etl.log 2&gt;&amp;1

# Nightly training at 1:10am
10 1 * * * cd /path/to/project &amp;&amp; /path/to/venv/bin/python
jobs/train_model.py &gt;&gt; logs/train.log 2&gt;&amp;1

# Inference every 5 minutes
*/5 * * * * cd /path/to/project &amp;&amp; /path/to/venv/bin/python
jobs/run_inference.py &gt;&gt; logs/infer.log 2&gt;&amp;1
```

In cron syntax, the five fields represent minute, hour, day-of-month, month, and day-of-week.

Windows Scheduling Option

On Windows, you can use Task Scheduler to run the same commands on a repeating trigger. The important concept is unchanged: a scheduler runs your scripts automatically at the times you define.

Alternative Scheduling for the Vibe-Coded App

Later, in the app-building section, you will see an additional scheduling option that can be easier for student projects: running scheduled jobs inside the application process using a lightweight job runner. This is useful for demos and learning, but OS-level scheduling is still the most reliable pattern.

- **Node/JavaScript**: node-cron (simple) or a background worker (more robust).
- **Python**: APScheduler (runs scheduled Python functions or commands).
- **ASP.NET/C#**: Quartz.NET or Hangfire (background jobs with dashboards).

Your web application does not retrain the model. Instead, it reads predictions written into shop.db and uses them to prioritize orders.

A simple “warehouse priority list” query can sort by late_delivery_probability descending and show the top orders to fulfill first.

17.8Vibe Code Priority Queue

What the App Needs

At this point, training produces a model file and metrics. In a production application, the next step is to turn predictions into an operational workflow feature. Here, that feature is a warehouse *priority queue*: orders with the highest predicted probability of late delivery rise to the top so the warehouse can process them first.

To keep the app simple, we treat the operational database as the system of record and write predictions back to it. The app then reads a single query (or a database view, if you choose to create one) to display the priority queue. The key deployment idea is that the app does not run machine learning code—it just reads predictions like any other table.

Single Query the App Uses

The app can use the following SQL query to return a prioritized list of orders. This query assumes you have a table named *order_predictions* keyed by *order_id* with columns *late_delivery_probability*, *predicted_late_delivery*, and *prediction_timestamp*.

```
SELECT
  o.order_id,
  o.order_timestamp,
  o.total_value,
  o.fulfilled,
  c.customer_id,
  c.first_name || ' ' || c.last_name AS customer_name,
  p.late_delivery_probability,
  p.predicted_late_delivery,
  p.prediction_timestamp
FROM orders o
JOIN customers c
  ON c.customer_id = o.customer_id
JOIN order_predictions p
  ON p.order_id = o.order_id
WHERE o.fulfilled = 0
ORDER BY
  p.late_delivery_probability DESC,
  o.order_timestamp ASC
```

```
LIMIT 50;
```

Operationally, this becomes the “next orders to pull” list. The only thing the UI must do is run this query and render the result in a table.

Using Cursor to Build the UI Feature

You will use Cursor (Education) to generate most of the application scaffolding. Your job is to provide the AI agent with clear requirements and a stable database contract. Use one of the three stacks below, based on your prior coding experience.

- If you have little or no web development background, use the *JavaScript stack* (Next.js) for the most guided path.
- If you are strongest in Python, use the *Python stack* (FastAPI) to keep everything in one language.
- If you have C# experience, use the *ASP.NET stack* (minimal APIs or MVC) for a professional enterprise-style pattern.

AI Prompts Students Can Paste

Pick one prompt below and paste it into Cursor (or Claude Code). After the agent generates code, you will run the app, validate the priority page loads, and confirm it returns the same rows as the SQL query above.

Option A: JavaScript (Next.js) Prompt

```
You are building a small student project web app using Next.js (App Router) and a SQLite database named &amp;quot;shop.db&amp;quot;.
Requirements:
1. Create a page at /warehouse/priority that displays a &amp;quot;Late Delivery Priority Queue&amp;quot; table.
2. The page must run this SQL query against shop.db and render the results:

SELECT
```

```
        o.order_id,
        o.order_timestamp,
        o.total_value,
        o.fulfilled,
        c.customer_id,
        c.first_name || ' ' || c.last_name AS customer_name,
        p.late_delivery_probability,
        p.predicted_late_delivery,
        p.prediction_timestamp
      FROM orders o
      JOIN customers c ON c.customer_id = o.customer_id
      JOIN order_predictions p ON p.order_id = o.order_id
      WHERE o.fulfilled = 0
      ORDER BY p.late_delivery_probability DESC, o.order_timestamp ASC
      LIMIT 50;

      3. Use a lightweight SQLite library for Node (better-sqlite3 preferred).
      4. Create a simple layout with a header, a short explanatory paragraph, and a table
(sortable columns optional).
      5. Include minimal styling (clean, readable). No authentication required.

      Deliverables:
      - All code changes needed in the Next.js project
      - Any install commands (npm) and how to run
      - A short note explaining where shop.db should be located in the repo
```

Option B: Python (FastAPI) Prompt

```
      You are building a small student project web app using Python FastAPI and Jinja2
templates with a SQLite database named &amp;quot;shop.db&amp;quot;.
      Requirements:
      1. Create a route GET /warehouse/priority that renders an HTML page titled
&amp;quot;Late Delivery Priority Queue&amp;quot;.
      2. The route must run this SQL query against shop.db and render the results in an
HTML table:

      SELECT
        o.order_id,
        o.order_timestamp,
        o.total_value,
        o.fulfilled,
        c.customer_id,
        c.first_name || ' ' || c.last_name AS customer_name,
        p.late_delivery_probability,
        p.predicted_late_delivery,
        p.prediction_timestamp
      FROM orders o
      JOIN customers c ON c.customer_id = o.customer_id
      JOIN order_predictions p ON p.order_id = o.order_id
      WHERE o.fulfilled = 0
      ORDER BY p.late_delivery_probability DESC, o.order_timestamp ASC
      LIMIT 50;

      3. Use the built-in sqlite3 module (no ORM).
      4. Provide minimal styling and a clean table layout.
      5. Include instructions to run with uvicorn.

      Deliverables:
      - Project structure (main.py, templates, static if needed)
      - pip install commands
```

```
- How to run and where to place shop.db
```

Option C: ASP.NET/C# Prompt

```
You are building a small student project web app using ASP.NET Core (minimal API or
MVC) and SQLite database &amp;quot;shop.db&amp;quot;.
Requirements:
1. Create an endpoint /warehouse/priority that returns an HTML page titled
&amp;quot;Late Delivery Priority Queue&amp;quot;.
2. The endpoint must run this SQL query against shop.db and render results in a
table:

SELECT
  o.order_id,
  o.order_timestamp,
  o.total_value,
  o.fulfilled,
  c.customer_id,
  c.first_name || ' ' || c.last_name AS customer_name,
  p.late_delivery_probability,
  p.predicted_late_delivery,
  p.prediction_timestamp
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_predictions p ON p.order_id = o.order_id
WHERE o.fulfilled = 0
ORDER BY p.late_delivery_probability DESC, o.order_timestamp ASC
LIMIT 50;

3. Use Microsoft.Data.Sqlite to query (no Entity Framework required).
4. Provide minimal CSS styling and clear layout.
5. Provide commands to run locally (dotnet run) and where shop.db should live.

Deliverables:
- Full code changes (Program.cs plus any views/templates if used)
- NuGet package install commands
- Run instructions
```

What to Check

- The page loads without errors and the table renders.
- The top rows have the highest *late_delivery_probability*.
- The result matches running the same SQL directly in a SQLite browser.

In the next section, you will run inference on new orders and write predictions into *order_predictions* so the app can display this priority queue.

17.9Vibe Code App

Goal

In this chapter, you built a realistic ML pipeline that reads from a live operational database, creates an analytical “warehouse” table for modeling, trains a model, saves a model artifact, generates predictions, and writes those predictions back to the operational database.

In this section, you will *vibe code* a complete (but simple) web app on top of that database. You will use an AI coding agent (Cursor or Claude Code) to generate most of the application scaffolding. Your job is to (1) provide clear requirements, (2) keep the database contract stable, and (3) test the application until it matches expected behavior.

To keep scope manageable, this app intentionally ignores authentication. Instead, it lets the user select an existing customer to “act as” during testing.

What Your App Must Do

- Use an existing SQLite database file named *shop.db* (operational DB).
- Provide a “Select Customer” screen (no signup/login).
- Allow placing a new order for the selected customer.
- Save the order + line items into *shop.db*.
- Show an order history page for that customer.
- Show the warehouse “Late Delivery Priority Queue” page (top 50).
- Provide a “Run Scoring” button that triggers the inference job and then refreshes the priority queue.

You will build the app in one of three stacks. The recommended default is JavaScript (Next.js) because it is widely supported by AI coding agents and has a straightforward developer experience.

Database Contract

Your AI agent must not invent new tables. It should only use the operational database tables you already have (for example, *customers*, *orders*, *order_items*, *products*, and *order_predictions*). If your database uses different table or column names, update the prompts below to match your schema.

The pipeline writes predictions into *order_predictions* keyed by *order_id*. The application should treat that table like any other application table.

Recommended Stack

For students with limited background, use: *Next.js + SQLite* for the web app, and a separate *Python inference script* that writes predictions into the database.

The rest of this section provides a complete sequence of copy/paste prompts. Paste them into Cursor (or Claude Code) in order. After each step, run the app and verify behavior before moving on.

Prompt 0: Project Setup (Next.js)

```
      You are generating a complete student project web app using Next.js (App Router) and
SQLite.
      Constraints:
      - No authentication. Users select an existing customer to &amp;quot;act as&amp;quot;.
      - Use a SQLite file named &amp;quot;shop.db&amp;quot; located at the project root (or
/data/shop.db if you prefer).
      - Use better-sqlite3 for DB access.
      - Keep UI simple and clean.
```

```
Tasks:
1. Create a new Next.js app (App Router).
2. Add a server-side DB helper module that opens shop.db and exposes helpers for
SELECT and INSERT/UPDATE using prepared statements.
3. Create a shared layout with navigation links:
  - Select Customer
  - Customer Dashboard
  - Place Order
  - Order History
  - Warehouse Priority Queue
  - Run Scoring
4. Provide install/run instructions (npm) and any required scripts.

Return:
- All files to create/modify
- Any commands to run
```

Prompt 0.5: Inspect the Database Schema

```
Add a developer-only page at /debug/schema that prints:
- All table names in shop.db
- For each table, the column names and types (PRAGMA table_info)

Purpose: Students can verify the real schema and adjust prompts if needed.
Keep it simple and readable.
```

Prompt 1: Select Customer Screen

```
Add a &amp;quot;Select Customer&amp;quot; page at /select-customer.

Requirements:
1. Query the database for customers:
  - customer_id
  - first_name
  - last_name
  - email
2. Render a searchable dropdown or simple list. When a customer is selected, store
customer_id in a cookie.
3. Redirect to /dashboard after selection.
4. Add a small banner showing the currently selected customer on every page (if
set).

Deliver:
- Any new routes/components
- DB query code using better-sqlite3
- Notes on where customer_id is stored
```

Prompt 2: Customer Dashboard

```
Create a /dashboard page that shows a summary for the selected customer.

Requirements:
1. If no customer is selected, redirect to /select-customer.
2. Show:
  - Customer name and email
  - Total number of orders for the customer
  - Total spend across all orders (sum total_value)
  - A small table of the 5 most recent orders (order_id, order_timestamp, fulfilled,
total_value)
3. All data must come from shop.db.

Deliver:
- SQL queries used
- Page UI implementation
```

Prompt 3: Place Order Page

```
Create a /place-order page that allows creating a new order for the selected
customer.

Requirements:
1. If no customer selected, redirect to /select-customer.
2. Query products (product_id, product_name, price) and let the user add 1+ line
items:
  - product
  - quantity
3. On submit:
  - Insert a row into orders for this customer with fulfilled = 0 and
order_timestamp = current time
  - Insert corresponding rows into order_items
  - Compute and store total_value in orders (sum price*quantity)
4. After placing, redirect to /orders and show a success message.

Constraints:
- Use a transaction for inserts.
- Keep the UI minimal (a table of line items is fine).

Deliver:
- SQL inserts
- Next.js route handlers (server actions or API routes)
- Any validation rules
```

Prompt 4: Order History Page

```
Create a /orders page that shows order history for the selected customer.

Requirements:
1. If no customer selected, redirect to /select-customer.
2. Render a table of the customer's orders:
  - order_id, order_timestamp, fulfilled, total_value
```

```
3. Clicking an order shows /orders/[order_id] with line items:
  - product_name, quantity, unit_price, line_total
4. Keep it clean and readable.

Deliver:
- The two pages
- SQL queries
```

Prompt 5: Warehouse Priority Queue Page

```
Create /warehouse/priority page that shows the &amp;quot;Late Delivery Priority
Queue&amp;quot;.

Use this SQL query exactly (adjust table/column names only if they differ in
shop.db):

SELECT
  o.order_id,
  o.order_timestamp,
  o.total_value,
  o.fulfilled,
  c.customer_id,
  c.first_name || ' ' || c.last_name AS customer_name,
  p.late_delivery_probability,
  p.predicted_late_delivery,
  p.prediction_timestamp
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_predictions p ON p.order_id = o.order_id
WHERE o.fulfilled = 0
ORDER BY p.late_delivery_probability DESC, o.order_timestamp ASC
LIMIT 50;

Requirements:
- Render the results in a table.
- Add a short explanation paragraph describing why this queue exists.

Deliver:
- Page code
```

Prompt 6: Run Scoring Button (Triggers Python Inference Job)

To keep the application simple, the web app will not run ML code. Instead, it triggers a Python inference script that writes predictions into *order_predictions*. The app then reloads the priority queue.

```
Add a /scoring page with a &amp;quot;Run Scoring&amp;quot; button.

Behavior:
```

```
1. When clicked, the server runs:
  python jobs/run_inference.py
2. The Python script writes predictions into order_predictions keyed by order_id.
3. The UI shows:
  - Success/failure status
  - How many orders were scored (parse stdout if available)
  - Timestamp

Constraints:
- Provide safe execution: timeouts and capture stdout/stderr.
- The app should not crash if Python fails; show an error message.
- Do not require Docker.

Deliver:
- Next.js route/handler for triggering scoring
- Implementation details for running Python from Node
- Any UI components needed
```

Prompt 7: Polishing and Testing Checklist

```
Polish the app for student usability and add a testing checklist.

Tasks:
1. Add a banner showing which customer is currently selected.
2. Add basic form validation on /place-order.
3. Add error handling for missing DB, missing tables, or empty results.
4. Provide a manual QA checklist:
  - Select customer
  - Place order
  - View orders
  - Run scoring
  - View priority queue with the new order appearing (after scoring)

Deliver:
- Final code changes
- A README.md with setup and run steps
```

Alternative Stack Prompts

If you prefer Python or C#, you can use the prompts below instead. These prompts generate the same app features but with different frameworks.

Option B: Python (FastAPI) Full-App Prompt

```
Build a complete student web app using Python FastAPI, Jinja2 templates, and SQLite shop.db (at project root).
No authentication: users select an existing customer to &amp;quot;act as&amp;quot;.
```

```
      Pages:
      - GET /select-customer: list/search customers and store customer_id in a cookie
      - GET /dashboard: summary stats for selected customer
      - GET/POST /place-order: select products + quantities and insert orders +
order_items
      - GET /orders: order history
      - GET /orders/{order_id}: order details with line items
      - GET /warehouse/priority: priority queue table using order_predictions
      - POST /scoring/run: runs python jobs/run_inference.py and then redirects to
/warehouse/priority

      Constraints:
      - Use sqlite3 (no ORM).
      - Use transactions for writes.
      - Provide minimal CSS.
      - Include a README with setup and run instructions (uvicorn).

      Deliver all code files and commands.
```

Option C: ASP.NET/C# Full-App Prompt

```
      Build a complete student web app using ASP.NET Core and SQLite shop.db (at project
root).
      No authentication: users select an existing customer to &amp;quot;act as&amp;quot; and store
customer_id in a cookie.

      Pages/Endpoints:
      - /select-customer (GET + POST): choose customer
      - /dashboard (GET): customer summary + recent orders
      - /place-order (GET + POST): create an order and order_items using a DB transaction
      - /orders (GET): order history
      - /orders/{orderId} (GET): order detail with line items
      - /warehouse/priority (GET): late delivery priority queue (join
orders/customers/order_predictions)
      - /scoring/run (POST): execute python jobs/run_inference.py and return status

      Constraints:
      - Use Microsoft.Data.Sqlite (no EF required).
      - Render simple HTML (Razor Pages or MVC ok).
      - Provide commands to run (dotnet run) and setup instructions.

      Deliver all code files, NuGet packages, and commands.
```

Key Idea

This app is intentionally simple, but it demonstrates a complete end-to-end pattern: operational data → analytics pipeline → trained model file → automated scoring → operational workflow improvement.

17.10Practice

In this chapter, you built and deployed an end-to-end machine learning pipeline to predict *late delivery* and integrate those predictions directly into an application workflow.

In this practice section, you will extend that pipeline to support an additional predictive task. You may either build a second pipeline from scratch or augment your existing pipeline to handle multiple targets.

Choose a Prediction Target

Select *one* of the following targets to model:

- *is_fraud*: a binary classification task focused on identifying potentially fraudulent orders.
- *risk_score*: a regression task that estimates overall order risk on a continuous scale.

Both targets already exist in the database and were generated using meaningful relationships. Your goal is not to discover a “perfect” model, but to practice designing a clean, deployable pipeline.

Pipeline Design Options

You may approach this practice in one of two ways:

- Build a *separate pipeline* with its own ETL script, training script, and model artifact.
- Extend your existing pipeline to support *multiple targets* using shared feature engineering and separate models.

Both approaches are valid. Choose the one that best matches your comfort level and time constraints.

Required Components

Your solution must include the following elements:

- An ETL step that produces a modeling-ready table in the analytical database.
- Explicit feature selection and a clearly defined target.
- A scikit-learn *Pipeline* that combines preprocessing and modeling.
- Train/test evaluation with appropriate metrics (classification or regression).
- Saved model artifact plus metadata and metrics files.
- Predictions written back to the operational database in a new table keyed by *order_id*.

To avoid naming confusion, use a table name such as *order_predictions_fraud* or *order_predictions_risk*, and include a prediction timestamp column.

Evaluation Guidance

For *is_fraud*, accuracy can be misleading, so include at least precision, recall, and F1 (or PR AUC if you know how). For *risk_score*, include at least MAE (and optionally RMSE).

Application Integration

Decide how your predictions would be used by the application:

- *Fraud prediction*: flag orders for manual review, delayed fulfillment, or additional verification.
- *Risk score*: sort or filter orders by risk level, or combine risk with delivery priority.

You do not need to fully implement the UI changes, but you should be able to explain how the predictions would change system behavior.

AI-Assisted Development

You are encouraged to use Cursor or Claude Code to generate portions of your pipeline. However, you remain responsible for:

- Defining the target and features clearly.
- Ensuring training and inference use identical preprocessing.
- Validating outputs and checking for leakage or overfitting.

Reflection Questions

After completing the exercise, answer the following:

- Which parts of the pipeline were reusable across targets?
- Where did target-specific logic belong?
- Would you deploy this model in a real system? Why or why not?

This practice reinforces a core deployment lesson: scalable ML systems are built from reusable pipelines, not one-off notebooks.

17.11Assignment

Complete the assignment below: